ArtBench-10 Generative Modeling: A Comparative Study of VAE, GAN, and Diffusion Models

Tiago Silva and João Coelho

University of Coimbra

Abstract. Nothing yet.

1 Introduction

2 Problem Formulation

3 Methodological Framework

3.1 Implementation Details and Reproducibility

To ensure reproducibility, we adopted a deterministic seeding policy across the pipeline. In the base training workflow, all stochastic operations were initialized with seed 42. During hyperparameter optimization, each Optuna trial used a deterministic trial-specific seed ($42 + \text{trial_id}$), and the train/validation split was generated with a fixed seed. Final evaluation was conducted across 10 distinct seeds, followed by reporting mean and standard deviation of metrics. During the experimentation phase, a 20% subset (10,000 images) was used for hyperparameter sweeps via Bayesian optimization. Once optimal parameters werei dentified, the final models were retrained on the full 50,000-image training set.

Regarding data preprocessing, all images were resized and center-cropped to 32×32, converted to tensors, and normalized channel-wise with mean $(0.5, 0.5, 0.5)$ and standard deviation $(0.5, 0.5, 0.5)$, resulting in inputs scaled to $[-1, 1]$. This normalization is important in our setting because it aligns the input scale with the output range typically used by generative architectures (e.g, tanh outputs values in this range), improves optimization conditioning, and contributes to more stable gradient dynamics, particularly for adversarial and diffusion-based training (forward noising process is defined around 0 mean Gaussian noise).

All training functions followed the same execution template:

- Initialization of models, output directories, and training variables;
- Main training loop for 50 epochs, with checkpoints and generated samples saved every 10 epochs;
- Saving of the final model.

Variational Autoencoder For this model, we implemented a symmetric 4 layer convolutional design (32, 16, 8, 4, 2) with leak Relu activations and batch normalization after each convolutional layer. Following [3] we also implemented the reparameterization trick to allow backpropagation through the stochastic sampling of the latent space.

Generative Adversarial Network For the adversarial baseline, we implemented a Deep Convolutional GAN (DCGAN) with a transposed-convolution generator and a strided-convolution discriminator just as dicussed in the PL classes with some notations coming from [5]. The generator maps a latent vector z to a $32 \times 32 \times 3$ image through four upsampling blocks (ConvTranspose2d + BatchNorm + ReLU), followed by a `tanh` output layer. The discriminator mirrors this design with four convolutional blocks using LeakyReLU activations and BatchNorm in intermediate layers, ending in a sigmoid probability for binary real/fake classification.

Training follows the standard alternating minimax procedure with binary cross-entropy loss: the discriminator is first updated using real images and generated images, and then the generator is updated to maximize the discriminator's probability of classifying generated samples as real. We used Adam optimization (with tuned learning rate and β_1, and fixed $\beta_2 = 0.999$), as well as one-sided label smoothing for real targets to reduce discriminator overconfidence and improve early training stability.

Latent Denoise Network The Latent Denoise Network model's architecture was the same as the one in the PL classes. For the diffusion process, the DDPM scheduler used was the GaussianDiffusion model. For the time steps embeddings, a Sinusoidal Position Embedding, with another 3 layers of Linear, SiLu and Linear transformations, respectivelly, were used. For the reverse diffusion step, ResnetBlocks in sequence were used. For the encoding and decoding of samples to and from the latent space, the VAE previously trained was used.

The only changes made to the provided code were to the number of latent channels, which had to be the same as the trained VAE latent channels', and the implementation of the sample, train step and image generation functions inside the class of the model. These sample and train step functions were implemented just like the respective PL class demanded.

Pixel UNet The Pixel UNet model's architecture, just like the previous model, was the same as the one in the Diffusion models's PL class. The diffusion process and the time steps embeddings were the same as the ones in the previous model. The only differences were in the reverse diffusion step, where 6 ResnetBlocks were used (2 in downscaling, 2 keeping the resolution and 2 in upscaling), and the lack of a latent space.

Similarly to the Latent Denoise Network, the main changes to the code were with the implementation of the sample, train step and image generation inside the class of the model. These functions are very similar to the ones implemented with the previous model, only with slight adaptations. The only other difference from the PL class's code was the change in the input channels, which became 3, due to the dataset images being represented in the RGB space.

3.2 Theoretical Rationale for Hyperparameter Choices

The choice of which hyperparameters to tune and the search space design follows 4 main principles: enable sufficient representational capacity to model ArtBench-10's complexity, stable training dynamics, regularization to prevent overfitting, and computational feasibility under resource constraints.

Variational Autoencoders (VAE) For the VAE we tuned latent dimensionality, base channels, learning rate, batch size, and β (the weight of the KL divergence term in the loss). Latent dimension is intended to control the capacity of capturing the underlying data distribution, expressing a trade-off between reconstruction fidelity and generalization. Base channels determine the width of the convolutional layers, which affects the model's ability to learn complex features. Learning rate was sampled from a log-uniform distribution to allow exploration across several orders of magnitude, which is crucial for stable convergence. The β parameter was also sampled from a log-uniform distribution to explore a wide range of regularization strengths, consistent with the objective function:

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta \cdot D_{KL}(q(z|x)||p(z)) \tag{1}$$

where β controls the trade-off between reconstruction accuracy and latent space regularization.

Generative Adversarial Networks (GAN) For GANs, the search space targeted parameters known to affect training stability and sample diversity. The latent dimensionality and the number of feature maps determine generator and discriminator capacity. We additionally tuned the real-label target used in label smoothing, which can reduce discriminator overconfidence and improve gradient quality early in training. The Adam β_1 parameter was also tuned because adversarial optimization is sensitive to momentum settings.

Diffusion Models For diffusion models, we tuned the denoiser width and depth (`model_channels` and `num_res_blocks`), learning rate, batch size, and the number of diffusion timesteps. The timestep count captures the trade-off between computational cost and generative fidelity: more timesteps generally improve sample quality but increase training and sampling cost. In the latent-diffusion setting, the VAE was kept frozen so that the study isolates the effect of denoiser hyperparameters and avoids conflating them with representation learning in the autoencoder.

4 Experimental Design

4.1 Hyperparameter Optimization Strategy

We performed hyperparameter optimization with Bayesian optimization (Optuna). To keep the search feasible on standard hardware, tuning was conducted on a development subset comprising 20% of the training data. For each model family, candidate configurations were trained on a training split and selected using performance on a held-out validation split. We minimized the following surrogate objectives: (i) *VAE*: validation reconstruction loss; (ii) *GAN*-based models: validation sum of discriminator and generator losses; (iii) *Diffusion models*: validation denoising loss under a fixed Gaussian noise schedule. The hyperparameter search spaces are summarized in Table 1, and the best configurations found on the studies are reported in Table 2. We used validation losses as surrogates during tuning because FID/KID are computationally expensive and high-variance at the per-trial scale: they require generating large sample sets and repeated feature extraction. In contrast, validation losses are available at each epoch and exhibit lower variance which improves sample efficiency under limited computational resources. We note, however, that GAN losses are far from beeing perfect proxies for perceptual sample quality; therefore, we use this objective only for hyperparameter selection during tuning and rely on qualitative inspection for the selected final parameters.

Table 1. Hyperparameter search spaces used in Bayesian optimization.

Model	**Parameters**	**Values / Range**
VAE	latent_dim, base_channels, batch_size	$\{16, 32, 64\}$; $\{16, 32, 64\}$; $\{32, 64, 128\}$
	lr, β	$[10^{-4}, 3 \times 10^{-3}]$ log-uniform; $[10^{-6}, 10^{-3}]$ log-uniform
DCGAN	latent_dim, feature_maps, batch_size	$\{128, 256, 512\}$; $\{32, 64\}$; $\{32, 64, 128\}$
	lr, β_1, real_label	$[10^{-4}, 5 \times 10^{-4}]$ log-uniform; $[0.3, 0.7]$; $[0.85, 1.0]$
PixelUNet	model_channels, batch_size, num_timesteps	$\{32, 64, 96\}$; $\{16, 32, 64\}$; $\{500, 1000\}$
	lr	$[10^{-4}, 5 \times 10^{-4}]$ log-uniform
LatentDenoiser	model_channels, num_res_blocks, batch_size	$\{32, 64, 96\}$; $\{2, 3, 4\}$; $\{16, 32, 64\}$
	lr, num_timesteps	$[10^{-4}, 5 \times 10^{-4}]$ log-uniform; $\{500, 1000\}$

4.2 Evaluation Protocol

Across the selected configurations (Table 2), the optimization process consistently favored smaller batch sizes. This aligns with the well-documented generalization gap observed in large-batch training [2]. Specifically, the inherent noise in small-batch stochastic gradients serves as a form of implicit regularization, facilitating the escape from sharp local minima and leading the model toward flatter, more robust regions of the loss landscape. Regarding the learning rates, the observed values are relatively low, consistent with the need for stable convergence and stability during training, especially for GANs which are known to be sensitive to hyperparameter settings. Latent dimensions reveal important differences across model families: the VAE favored a much smaller latent space (32) compared to the DCGAN (256), which may reflect the different in the model's architectures: GANs don't pay an explicity penalty (KL divergence) for using larger latent spaces, while VAEs are encouraged to find compact representations. A larger latent space may encourage more diverse samples as long as the generator actually uses it. For the VAE, the optimal β value is very low, indicating that the model prioritizes reconstruction fidelity over latent space regularization. For the DCGAN, the optimal β_1 value is around 0.48, which is close to the commonly used value of 0.5 [5], suggesting a balance between stability and convergence speed. The real_label parameter being close to 1 (0.9836) indicates that the discriminator benefits from a small anti-overconfidence margin, which can help prevent it from becoming too confident and overpowering the generator during training without weakening the learning signal too much.

Table 2. Best configurations obtained.

Parameter	**VAE**	**DCGAN**	**PixelUnet**	**LatentDenoiser**
Best objective value	0.02658	1.68158	0.0358	0.2598
Learning rate	0.0005317	0.0001261	0.0004748	0.0001769
Batch size	32	32	16	32
Latent dimension	32	256	64	64
Base channels / feature maps	64	64	64	64
β (VAE)	1.14×10^{-6}	–	–	–
β_1 (Adam, DCGAN)	–	0.4813	–	–
real_label (DCGAN)	–	0.9836	–	–
num_timesteps (diffusion)	–	–	1000	1000
num_res_blocks (LatentDenoiser)	–	–	–	4

5 Results and Interpretation

5.1 Quantitative Results

Table 3 summarizes the quantitative evaluation across model families using 5,000 generated samples and 10 random seeds per model. We report mean ± standard deviation across seeds. Arrows indicate whether lower (↓) or higher (↑) values are preferred. The best value in each metric is highlighted in bold.

Table 3. Summary of quantitative results (mean ± std across seeds).

Model	**FID ↓**	**KID ↓**	**Precision ↑**
VAE	134.228 ± 0.531	0.13136 ± 0.00109	0.80548 ± 0.00671
DCGAN	**101.248 ± 0.864**	**0.07664 ± 0.00153**	0.86400 ± 0.00489
PixelUNet	116.933 ± 0.708	0.09972 ± 0.00107	**0.90804 ± 0.00512**
LatentDenoiser	119.271 ± 0.416	0.10068 ± 0.00122	0.78472 ± 0.00466

Overall, DCGAN achieves the best performence in terms of global distributional alignment obtaining the lowest FID (101.248 ± 0.864) and KID (0.07664 ± 0.00153) scores. This suggests that the DCGAN is more effective at capturing the overall data distribution compared to the other models. The VAE, on the other hand, has the worst performance in terms of FID and KID, which is consistent with the known limitations of VAEs in generating overly smooth reconstructions due to the variational objective, which can lead to blurry samples and less realistic outputs. PixelUNet achieves the highest precision (0.90804 ± 0.00512), indicating that it generates a higher proportion of samples that are close to the real data manifold. However, the higher FID and KID values compared to DCGAN suggest that the improvement in sample realism does not necessarily translate to better overall distributional alignment. They, therefore, generate more realistic samples but with less diversity. The LatentDenoiser achieves intermediate FID and KID values but the lowest precision among the evaluated models. This suggests that while its generated distribution is moderately aligned with the target dataset at a global level, a larger fraction of its individual samples fall outside the real data manifold compared to the other architectures. Such behavior may indicate incomplete convergence or reduced stability in the denoising process during sampling. In addition, the small standard deviations across seeds for all models suggest that the results are consistent and not heavily influenced by random initialization or stochastic training dynamics which is a positive sign of the robustness of the training process.

6 Conclusion

Use of External Resources and AI Tools

This project utilized ArtBench-10 (Kaggle). Gemini assisted in LaTeX formatting and analysis verification.

References

1. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: Gans trained by a two time-scale update rule converge to a local nash equilibrium. In: Advances in Neural Information Processing Systems (NeurIPS). pp. 6626–6637 (2017)
2. Keskar, N.S., Mudigere, D., Nocedal, J., Smelyanskiy, M., Tang, P.T.P.: On large-batch training for deep learning: Generalization gap and sharp minima. CoRR **abs/1609.04836** (2016), `http://arxiv.org/abs/1609.04836`
3. Kingma, D.P., Welling, M.: Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114 (2013), `https://arxiv.org/abs/1312.6114`
4. Liao, P., Cheng, X., Sridhar, K., Li, Y., Phillip, I., Torralba, A.: The artbench dataset for fine-grained artistic style understanding and generation. arXiv preprint arXiv:2206.11404 (2022), `https://arxiv.org/abs/2206.11404`
5. Radford, A., Metz, L., Chintala, S.: Unsupervised representation learning with deep convolutional generative adversarial networks. In: International Conference on Learning Representations (ICLR) (2016), `https://arxiv.org/abs/1511.06434`